

### EXPERIMENT-10: Express REST API CRUD application

Develop an Express web application that can interact with REST APIs to perform CRUD operations on student data using Postman.

**Objective :** Build a RESTful student API with create/read/update/delete endpoints.

**Tools :** Node.js, Express, REST, Postman

**Outcome :** A student management REST API that supports full CRUD via HTTP requests.

#### Answer:

Define Express routes for student CRUD operations and use JSON body parsing to handle request data.

#### Folder structure:

Week-10/

├── server.js

├── public/

| └── index.html

#### Execution:

# Student REST API

A small Express app with a single-page student manager and JSON CRUD endpoints.

Student records are stored in memory, so they reset when the server restarts.

## Run

``powershell

npm install

npm start

``

Open `http://localhost:3000` for the page. Use the same API in Postman:

| Method | URL               | Purpose          |
|--------|-------------------|------------------|
| GET    | `/api/students`   | List students    |
| GET    | `/api/students/1` | Get one student  |
| POST   | `/api/students`   | Create a student |
| PUT    | `/api/students/1` | Update a student |

| DELETE | `/api/students/1` | Delete a student |

**File Name: server.js**

**Code:**

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;
const students = [];
let nextId = 1;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

function validateStudent(data) {
  const fields = ["name", "email", "course"];
  return fields.every(
    (field) => typeof data[field] === "string" && data[field].trim()
  );
}

app.get("/api/students", (req, res) => {
  res.json(students);
});

app.get("/api/students/:id", (req, res) => {
  const student = students.find((entry) => entry.id === Number(req.params.id));
  if (!student) return res.status(404).json({ error: "Student not found" });
  res.json(student);
});

app.post("/api/students", (req, res) => {
  if (!validateStudent(req.body)) {
    return res.status(400).json({ error: "Name, email, and course are required" });
  }
}
```

```

const student = {
  id: nextId++,
  name: req.body.name.trim(),
  email: req.body.email.trim(),
  course: req.body.course.trim(),
};
students.push(student);
res.status(201).json(student);
});

app.put("/api/students/:id", (req, res) => {
  const student = students.find((entry) => entry.id === Number(req.params.id));
  if (!student) return res.status(404).json({ error: "Student not found" });
  if (!validateStudent(req.body)) {
    return res.status(400).json({ error: "Name, email, and course are required" });
  }

  student.name = req.body.name.trim();
  student.email = req.body.email.trim();
  student.course = req.body.course.trim();
  res.json(student);
});

app.delete("/api/students/:id", (req, res) => {
  const studentIndex = students.findIndex(
    (entry) => entry.id === Number(req.params.id)
  );
  if (studentIndex === -1) {
    return res.status(404).json({ error: "Student not found" });
  }

  students.splice(studentIndex, 1);
  res.status(204).end();
});

app.listen(PORT, () => {

```

```
console.log(`Student app running at http://localhost:${PORT}`);
});
```

**File Name : public/index.html**

**Code:**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Student Manager</title>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; background: #f3f6f4; color: #202923; font: 16px/1.45 Arial, sans-serif; }
    main { width: min(900px, calc(100% - 32px)); margin: 38px auto; }
    h1 { margin: 0 0 4px; font-size: 28px; }
    .subtitle { margin: 0 0 24px; color: #59645d; }
    form { display: grid; grid-template-columns: repeat(3, 1fr) auto; gap: 10px; align-items: end;
      margin-bottom: 16px; }
    label { display: grid; gap: 5px; font-size: 14px; font-weight: 600; }
    input { width: 100%; min-height: 40px; padding: 8px 10px; border: 1px solid #bbc5be; border-
      radius: 4px; background: white; font: inherit; }
    button { min-height: 40px; padding: 8px 14px; border: 0; border-radius: 4px; background:
      #176b4b; color: white; font: inherit; cursor: pointer; }
    button:hover { background: #10563b; }
    button.secondary { background: #e1e8e3; color: #26372e; }
    button.delete { background: #a13f35; }
    .form-actions { display: flex; gap: 8px; }
    #message { min-height: 24px; margin: 0 0 8px; color: #46564c; }
    .table-wrap { overflow-x: auto; border: 1px solid #d5ddd7; border-radius: 4px; background:
      white; }
    table { width: 100%; border-collapse: collapse; text-align: left; }
    th, td { padding: 11px 12px; border-bottom: 1px solid #e3e8e4; }
    th { background: #eaf0ec; font-size: 14px; }
    tbody tr:last-child td { border-bottom: 0; }
    td.actions { white-space: nowrap; }
    td.actions button { min-height: 32px; margin-right: 5px; padding: 5px 10px; font-size: 14px; }
```

```
#empty { padding: 18px 12px; color: #647067; text-align: center; }
@media (max-width: 680px) {
  main { margin-top: 24px; }
  form { grid-template-columns: 1fr; }
  .form-actions button { flex: 1; }
  th, td { padding: 9px 8px; }
}
</style>
</head>
<body>
  <main>
    <h1>Student Manager</h1>
    <p class="subtitle">Create, view, update, and delete students.</p>

    <form id="student-form">
      <label>Name<input id="name" name="name" required></label>
      <label>Email<input id="email" name="email" type="email" required></label>
      <label>Course<input id="course" name="course" required></label>
      <div class="form-actions">
        <button id="submit-button" type="submit">Add student</button>
        <button id="cancel-button" class="secondary" type="button" hidden>Cancel</button>
      </div>
    </form>

    <p id="message" role="status" aria-live="polite"></p>
    <div class="table-wrap">
      <table>

        <thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Actions</th></tr>
        </thead>

        <tbody id="student-list"></tbody>
      </table>

      <div id="empty">No students yet.</div>
    </div>
  </main>
```

```

<script>
  const form = document.querySelector("#student-form");
  const list = document.querySelector("#student-list");
  const message = document.querySelector("#message");
  const empty = document.querySelector("#empty");
  const submitButton = document.querySelector("#submit-button");
  const cancelButton = document.querySelector("#cancel-button");
  let editingId = null;

  async function request(url, options) {
    const response = await fetch(url, options);
    if (!response.ok) {
      const result = await response.json();
      throw new Error(result.error || "Request failed");
    }
    return response.status === 204 ? null : response.json();
  }

  async function loadStudents() {
    try {
      const students = await request("/api/students");
      list.replaceChildren();
      empty.hidden = students.length > 0;
      for (const student of students) {
        const row = document.createElement("tr");
        for (const value of [student.id, student.name, student.email, student.course]) {
          const cell = document.createElement("td");
          cell.textContent = value;
          row.append(cell);
        }

        const actions = document.createElement("td");
        actions.className = "actions";
        const edit = document.createElement("button");
        edit.type = "button";
        edit.className = "secondary";
      }
    }
  }

```

```

edit.textContent = "Edit";
edit.addEventListener("click", () => startEditing(student));
const remove = document.createElement("button");
remove.type = "button";
remove.className = "delete";
remove.textContent = "Delete";
remove.addEventListener("click", () => deleteStudent(student.id));
actions.append(edit, remove);
row.append(actions);
list.append(row);
}
} catch (error) {
  message.textContent = error.message;
}
}

```

```

function startEditing(student) {
  editingId = student.id;
  form.elements.name.value = student.name;
  form.elements.email.value = student.email;
  form.elements.course.value = student.course;
  submitButton.textContent = "Save changes";
  cancelButton.hidden = false;
  message.textContent = `Editing student ${student.id}`;
  form.elements.name.focus();
}

```

```

function resetForm() {
  editingId = null;
  form.reset();
  submitButton.textContent = "Add student";
  cancelButton.hidden = true;
  message.textContent = "";
}

```

```

form.addEventListener("submit", async (event) => {

```

```

event.preventDefault();
const student = {
  name: form.elements.name.value.trim(),
  email: form.elements.email.value.trim(),
  course: form.elements.course.value.trim(),
};
const isEditing = editingId !== null;
try {
  await request(isEditing ? `/api/students/${editingId}` : "/api/students", {
    method: isEditing ? "PUT" : "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(student),
  });
  resetForm();
  message.textContent = isEditing ? "Student updated." : "Student added.";
  await loadStudents();
} catch (error) {
  message.textContent = error.message;
}
});

async function deleteStudent(id) {
  if (!window.confirm("Delete this student?")) return;
  try {
    await request(`/api/students/${id}`, { method: "DELETE" });
    if (editingId === id) resetForm();
    message.textContent = "Student deleted.";
    await loadStudents();
  } catch (error) {
    message.textContent = error.message;
  }
}

cancelButton.addEventListener("click", resetForm);
loadStudents();
</script>

```

</body>

</html>